

LGTM

Looks Good To Monitor

Created by: cgodard

Summary: This document is a subject on observability, Kubernetes, and distributed systems.

Version: 1.1

Contents

I Foreword **2**

II AI Instructions **3**

III Introduction **5**

IV General guidelines **6**
IV.0.1 The Vision 6
IV.0.2 Technical Requirements 6

V Mandatory part **8**
V.1 Part 1: The web application 8
V.1.1 Frontend: The Playground 8
V.1.2 Backend and WASM execution 9
V.1.3 Security requirements 11
V.1.4 IPFS integration: Share with the World 11
V.2 Part 2: The observability stack 13
V.2.1 Required components 13
V.2.2 Dashboard 1: Metrics - The Big Picture 13
V.2.3 Dashboard 2: Traces - The Detective's Tool 14

VI Readme Requirements **17**

VII Bonus part **18**

VIII Submission and peer-evaluation **19**

IX Credits **21**

Chapter I

Foreword

"It works on my machine!"

Five minutes later, production is down and nobody knows why.

You SSH in. You run `tail -f /var/log/app/.log | grep ^ERROR`. Lines scroll by. None of them tell you what broke, when it started, or how many users just saw a blank page.

Somewhere between the fifth grep and the third coffee, you start dreaming of a system that would simply... show you.

That system has a name: observability. Logs, metrics, and traces, working together so you never have to guess again.

LGTM - Because "Looks Good To Me" is what you say when you actually know what's going on.

Chapter II

AI Instructions

Context

AI is now a powerful coding partner — alongside your peers — for tackling large and demanding projects. You will guide it through both technical and non-technical aspects of your work.

AI tools can boost your efficiency and improve the quality of your output, but you should be able to dive deep into any part of the project without relying on them.

Your AI partner supports you, but you remain fully responsible for making informed technical decisions and to clearly explain and defend them.

Main message

- 👉 Strive for a mature and responsible use of AI.
- 👉 Never let AI take responsibility for decisions — especially when it lacks awareness of your goals, constraints, or team dynamics.
- 👉 Maintain creativity, innovation, and human oversight through active collaboration with your peers. AI is trained on existing data and rarely generates truly new ideas.
- 👉 Stay informed about emerging trends and be ready to adapt to new concepts and technologies.

Learner rules:

- Maintain intellectual leadership over your projects and make your own informed decisions.
- Prioritise the collective intelligence of your team and peers.
- Actively stay informed about the ongoing evolution of AI technologies.

Phase outcomes:

- AI engineering skills.
- Increased efficiency.
- Greater reliability and quality.
- A pioneering mindset.

Comments and examples:

- Your peers can identify trade-offs, question assumptions, and help you improve. The first answer from an AI might not be the best — it may lack efficiency, security, or real added value. Now more than ever, you should rely on your peers.
- AI can make you faster, but your peers make you better. Collaboration, discussion, and mutual challenge are key to success.
- Be transparent about how AI was used in your projects, and clearly identify what was generated by AI tools.

✓ Good practice:

> I asked AI to help generate unit tests for my API. I reviewed them with my teammate, and we adjusted them for edge cases. It saved time, and we both learned something new.

✗ Bad practice:

> I had AI generate the entire architecture of my project. It "works," but when I'm asked to explain design decisions during the peer review or in front from of a customer, I cannot. I lose credibility and I fail.

Chapter III

Introduction

Welcome to the fascinating world of observability! In this project, you will discover how modern applications are monitored in production environments.

Imagine building a playground where users can run code snippets safely using **WebAssembly**, share them across the globe via **IPFS**, and watch every step of execution unfold in beautiful dashboards. That's exactly what you're about to create.

The project combines cutting-edge technologies:

- **WASM/WASI** - Execute untrusted code safely in a sandboxed environment
- **IPFS** - Store and share code on a distributed filesystem
- **LGTM Stack** - Monitor everything with Loki, Grafana, Tempo, and Mimir
- **OpenTelemetry** - Instrument your application using industry standards
- **Kubernetes** - Orchestrate your entire infrastructure

By the end, you'll understand why observability is crucial for modern applications, how WebAssembly revolutionizes code execution, and how to implement both effectively.



`This project involves multiple complex technologies working together. Don't rush, take time to understand each component's role in the system.`

Chapter IV

General guidelines

IV.0.1 The Vision

Picture this: A developer writes a Rust function, clicks "Run", and within seconds sees not only the output, but also a complete trace showing compilation time (23ms), execution time (5ms), and IPFS upload time (142ms). Meanwhile, Grafana dashboards update in real-time, showing this was the 42nd successful execution today.

That's the power of observability combined with modern web technologies.

IV.0.2 Technical Requirements

- The entire project must be completed in a **virtual machine**. All tools and services mentioned below must run inside the virtual machine.
- You must use a local **Kubernetes** cluster (minikube, kind, k3d, etc.) to run the applications and services.
- You must provide a `Makefile` that installs the required packages and deploys your entire infrastructure.
- You may use any programming language with OpenTelemetry and IPFS support.
- Your web application must be accessible at `lgtm.local`.
- Grafana must be accessible at `grafana.lgtm.local`.
- IPFS gateway must be accessible at `ipfs.lgtm.local`.



`You may NOT use cloud services. Everything must run locally inside your virtual machine.`



You may substitute LGTM components (e.g., Jaeger for Tempo, Prometheus for Mimir) as long as Grafana remains the visualization tool.

Chapter V

Mandatory part

This project consists of building a complete web application with observability.



`Read the entire subject before starting.`

V.1 Part 1: The web application

Create a web application allowing users to write and execute code securely.

V.1.1 Frontend: The Playground

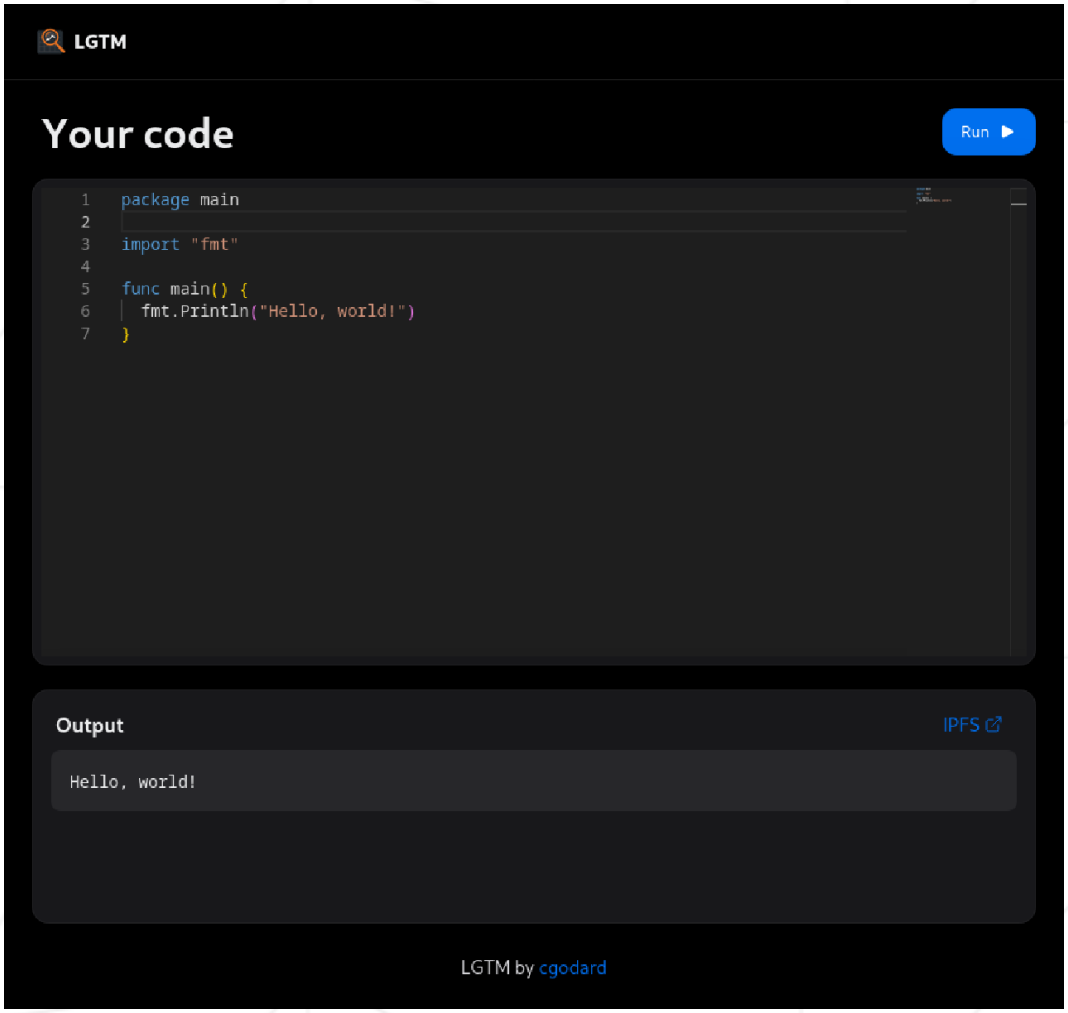
Imagine a sleek, modern interface where code comes to life. Your frontend is the gateway between users and the powerful backend infrastructure.

Your frontend must include:

- A code editor (Monaco, CodeMirror, or similar)
- A pre-loaded code example that demonstrates the language's capabilities
- A prominent "Run" button
- Visual feedback during execution (loading spinner, progress indicator)
- Clear error display when compilation or execution fails
- Beautiful output display when execution succeeds
- An IPFS link to share the code and its output with the world



Think about user experience. A developer should feel excited to try your platform, not intimidated by it.



V.1.2 Backend and WASM execution

Your backend must:

- Receive code via API
- Compile code to **WebAssembly** (WASM)
- Execute WASM in a **WASI** runtime
- Capture execution output
- Upload code and output to IPFS
- Return results to frontend

- Instrument all operations with OpenTelemetry



Choose a programming language that can compile to WASM. Popular choices include Rust, C/C++, AssemblyScript, or Go (with TinyGo). Your backend will need a WASI runtime like Wasmtime, Wasmer, or WasmEdge to execute the compiled WASM modules.

V.1.3 Security requirements

The beauty of WASM/WASI is its capability-based security model. Your implementation must enforce these restrictions:

- No network access (or whitelist only)
- No filesystem access (or whitelist only)
- Memory limit (e.g., 10MB)
- Time limit (e.g., 5 seconds)
- Output size limit (e.g., 10kB)



WASI provides sandboxing by default, but you must configure it properly. The runtime should not grant any capabilities unless explicitly needed.



If you choose not to use WASM/WASI, you must implement equivalent sandboxing through other means (seccomp, Landlock, containers, etc.) and justify your choice during evaluation.

V.1.4 IPFS integration: Share with the World

IPFS (InterPlanetary File System) is a distributed filesystem where content is addressed by its hash (CID). Think of it as Git for files - once uploaded, your code becomes part of a global, permanent web.

When a user runs code successfully, imagine this flow:

1. Code executes and produces output
2. Both are packaged into a directory
3. Directory is uploaded to IPFS, receiving a unique CID
4. User gets a link: `ipfs.lgtm.local/ipfs/QmXx...`
5. This same CID works on any IPFS gateway worldwide!

IPFS links must point to a directory containing the source code and output. This directory must be accessible via public gateways using the same CID.



IPFS is a peer-to-peer network: your computer needs to be reachable from the outside world for the network connectivity to work. If you are behind NAT without UPnP/IPv6, explain the limitations to your evaluators. Consider using a mobile network or a remote server during the defense.



Test your IPFS setup by uploading a file and accessing it from ipfs.io/ipfs/<your-cid> on a different device or network.

V.2 Part 2: The observability stack

Now comes the magic: making the invisible visible. Every code execution, every millisecond, every error - all captured, correlated, and visualized.

This is where you transform from a developer who *hopes* things work to one who *knows* exactly what's happening.

V.2.1 Required components

Your LGTM stack must include:

- **Grafana** - Your window into the system (visualization)
- **Loki** or equivalent - The story of what happened (logs)
- **Tempo** or Jaeger - The journey of each request (traces)
- **Mimir** or Prometheus - The numbers that matter (metrics)
- **OpenTelemetry Collector** or Alloy - The universal translator (telemetry)



These components work together like an orchestra. Each plays its part, but together they create a symphony of observability.

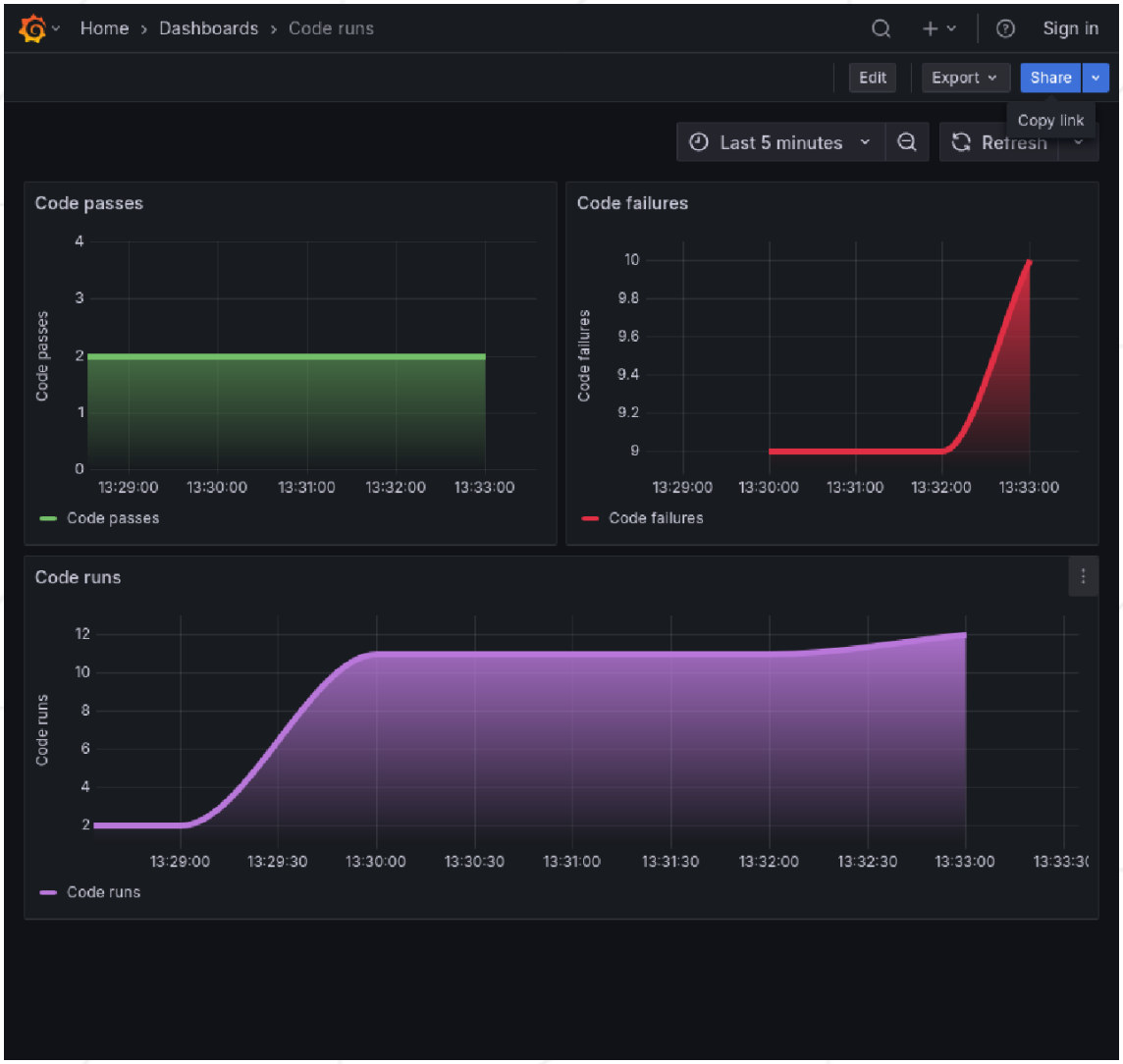
V.2.2 Dashboard 1: Metrics - The Big Picture

Imagine a dashboard that updates in real-time as users run code. Each execution increments counters, painting a picture of your platform's usage.

Create a Grafana dashboard showing:

- Number of successful executions (the green line going up!)
- Number of failed executions (debugging opportunities)
- Total executions (your platform's heartbeat)

Use the OpenTelemetry Metrics API to send data to Prometheus/Mimir.





Think about what other metrics would be valuable. Execution time distribution? Most popular languages? Peak usage hours?

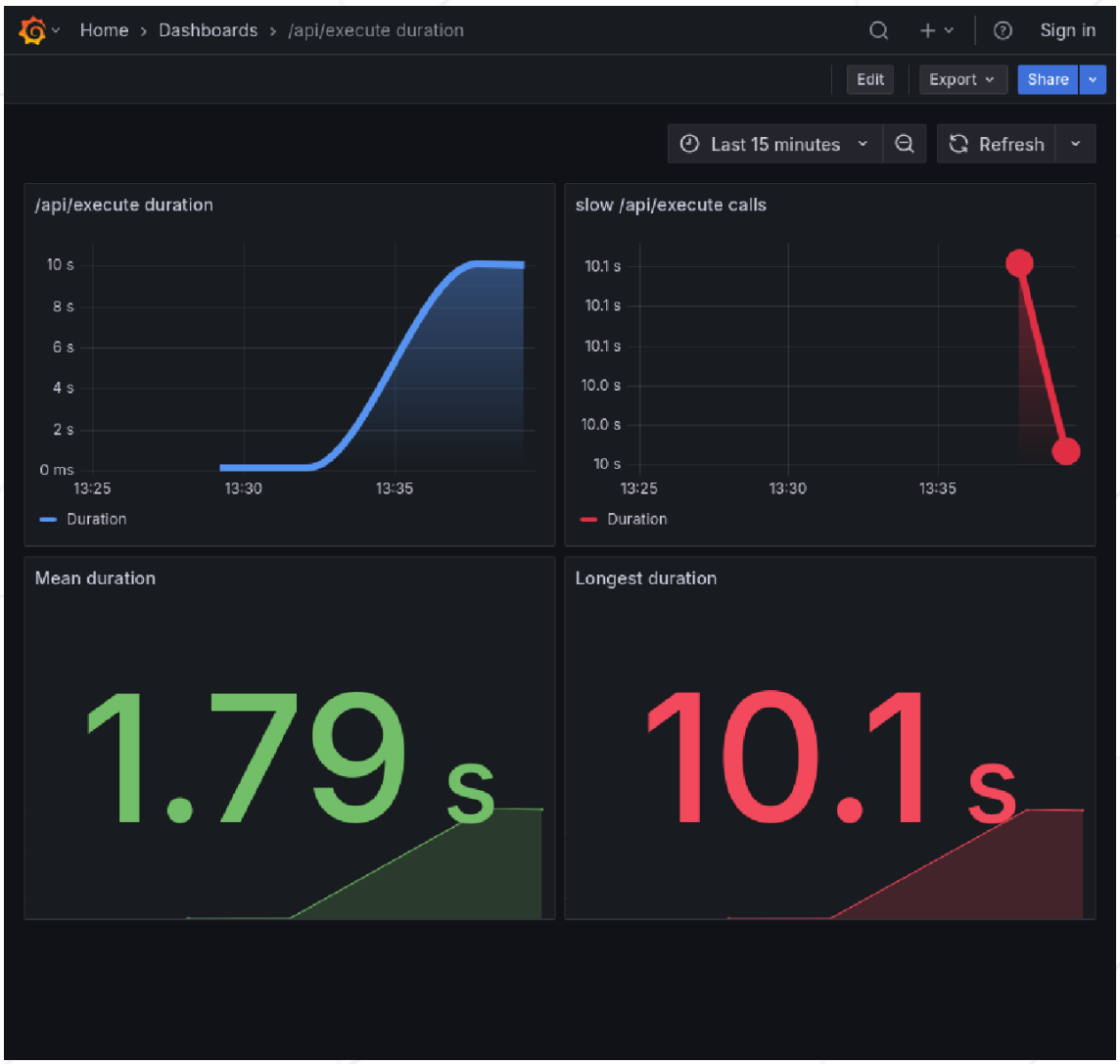
V.2.3 Dashboard 2: Traces - The Detective's Tool

Traces are like X-rays for your application. They show you exactly where time is spent, where errors occur, and how components interact.

Create a Grafana dashboard showing:

- Trace duration over time (spot the outliers!)
- Slowest executions (what's taking so long?)
- Mean execution duration (the baseline)

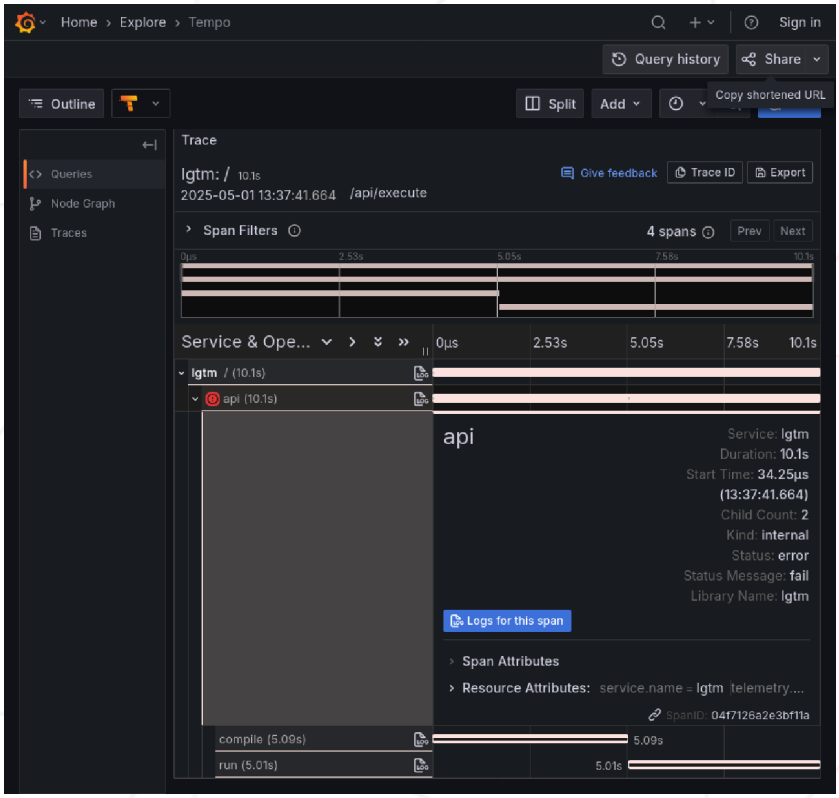
- Longest execution duration (the record holder)



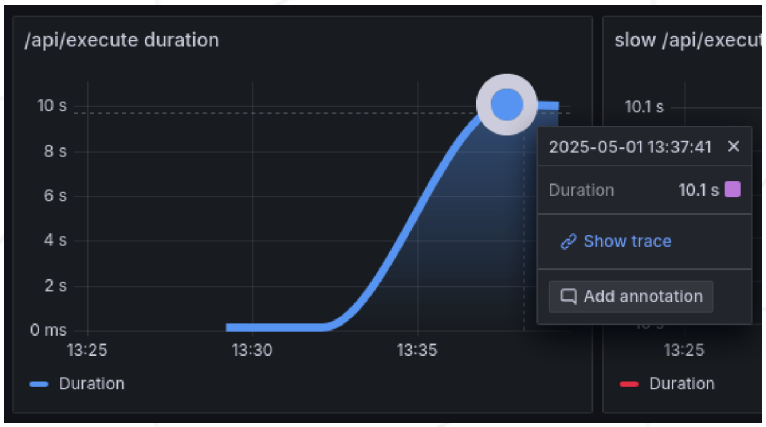
Each trace must include these spans:

- Root span (HTTP request) - The journey begins
- Compilation time (if applicable) - Turning code into WASM
- Execution time - The actual code running
- IPFS upload time - Sharing with the world

Use the OpenTelemetry Tracing API to send spans to Tempo/Jaeger.



Clicking "Logs for this span" must show correlated logs. Clicking a data point on the duration graphs must open the corresponding trace (exemplars) - a trace is only useful if you can actually reach it from the metric that flagged it.
Traces must indicate errors when they occur. Include a way to identify the executed code in traces (CID, hash, or snippet).





A well-instrumented trace tells a story. When something goes wrong at 3 AM, this trace will be your best friend.

Chapter VI

Readme Requirements

A `README.md` file must be provided at the root of your Git repository. Its purpose is to allow anyone unfamiliar with the project (peers, staff, recruiters, etc.) to quickly understand what the project is about, how to run it, and where to find more information on the topic.
The `README.md` must include at least:

- The very first line must be italicized and read: *This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]].*
- A "**Description**" section that clearly presents the project, including its goal and a brief overview.
- An "**Instructions**" section containing any relevant information about compilation, installation, and/or execution.
- A "**Resources**" section listing classic references related to the topic (documentation, articles, tutorials, etc.), as well as a description of how AI was used — specifying for which tasks and which parts of the project.

➠ **Additional sections may be required depending on the project** (e.g., usage examples, feature list, technical choices, etc.).

Any required additions will be explicitly listed below.

Chapter VII

Bonus part

Ready to take it to the next level? These bonuses will transform your project from impressive to extraordinary.

- **Deep IPFS Integration** - Link IPFS implementation traces with your application traces. See exactly how long it takes for content to propagate through the network.
- **Infrastructure Observability** - Add a Kubernetes cluster monitoring dashboard. Watch your pods, nodes, and resources in real-time.
- **Polyglot Platform** - Support multiple programming languages. Let users choose between Rust, C, Go, or AssemblyScript.
- **Social Sharing** - Allow code sharing via short URLs. Make it easy to share cool code snippets with friends.
- **Performance Profiling** - Add profiling data to traces. Show CPU usage, memory allocation, and hot paths.



Bonus evaluated only if mandatory part is flawless.



Each bonus should be production-quality. Don't just add features - make them shine!

Chapter VIII

Submission and peer-evaluation

Submit your work in your Git repository. Only repository content will be evaluated.

Your repository must contain:

- A `Makefile` that installs and deploys everything
- All Kubernetes manifests and configuration files
- Application source code (frontend and backend)
- The `README.md` with standard sections



Evaluation happens on the evaluated student's computer. Ensure your Makefile works on a newly created virtual machine.



You must explain all concepts used (observability, WASM, IPFS, OpenTelemetry, etc.) during evaluation.

Expected directory structure:

```
$> ls -la
drwxr-xr-x 10 user user  320 Jan 15 10:00 .
drwxr-xr-x  5 user user  160 Jan 15 09:00 ..
-rw-r--r--  1 user user  XXX Jan 15 10:00 .gitignore
-rw-r--r--  1 user user XXXX Jan 15 10:00 Makefile
-rw-r--r--  1 user user XXXX Jan 15 10:00 README.md
drwxr-xr-x  8 user user  256 Jan 15 10:00 backend
drwxr-xr-x  6 user user  192 Jan 15 10:00 frontend
drwxr-xr-x  4 user user  128 Jan 15 10:00 k8s
drwxr-xr-x  3 user user   96 Jan 15 10:00 dashboards
```



Test your Makefile thoroughly before submission.

Chapter IX

Credits

★ ✧ ★

A Word of Thanks

This project was born as a beta-test: a first version, submitted to a group of students before being made official for everyone.
We would like to warmly thank every student who took the time to work on it during that beta-test phase, and, most importantly, to report typos, inconsistencies, bugs, and suggestions along the way.
Their feedback made this subject clearer, fairer, and more solid. This project would not be what it is today without them.

☆ **Thank you!** ☆